System Implementation & Deployment

1.1 Overview:

This chapter describes the procedures required to set up the development environment and execute the Retail Order Management System. The implementation follows a full-stack architecture where the backend server and frontend client must be initialized correctly to establish a connection with the cloud database.

1.2 Environment Setup

Before running the application, the following steps must be completed:

1. **Dependency Installation:** Open the terminal in the project root directory and run npm install. This installs all necessary packages such as express, bcryptjs, jsonwebtoken, and mongodb.
2. **Database Configuration:** Ensure the MongoDB Atlas connection string is correctly placed in the server file to allow the application to store and retrieve data.

1.3 Execution Steps

The system requires two main components to be active simultaneously: the Backend API and the Frontend User Interface.

1.3.1 Starting the Backend Server

The backend acts as the engine of the application. To start the server:

1.3.1.1 Open the terminal or command prompt.

1.3.1.2 Navigate to the project folder.

1.3.1.3 Run the command: node server.js.

1.3.1.4 The terminal should display a message: **"Server is running on port 5500"** and **"Connected to MongoDB"**, confirming the backend is ready to handle requests.

```
D:\IIT\2.2\New folder\Project\retail-order-management-system\server>node server.js
Server running on http://localhost:8081
Connected to MongoDB
```

Figure 1.1: Running the server from Command Prompt

Figure 1.2: Confirming that server is running on localhost

1.3.2 Launching the Frontend

Since the frontend is built with Vanilla JavaScript and HTML, it can be launched using a local development server:

1.3.2.1 Open the index.html file in VS Code.

1.3.2.2 Click on the **"Go Live"** button (Live Server extension) at the bottom right of the editor.

1.3.2.3 The browser will automatically open the application at http://127.0.0.1:5500.

1.3.2.4 The frontend will immediately send a fetch request to the Node.js server to load the available products.

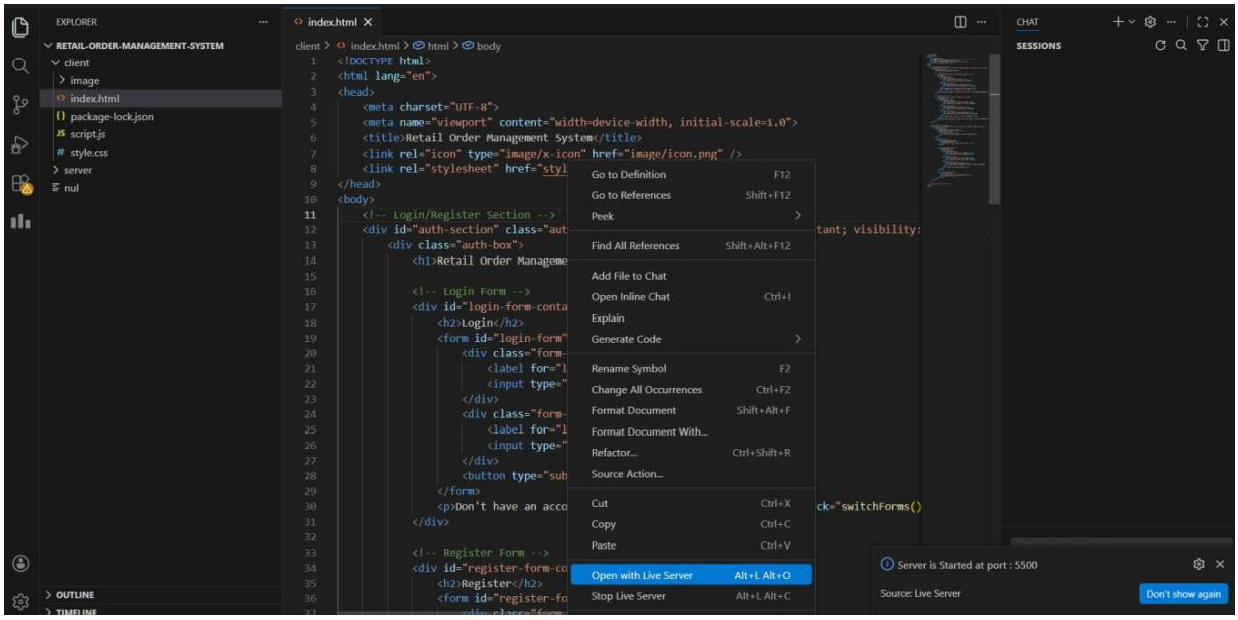

Figure 1.3: Opening the website with live server in VS Code

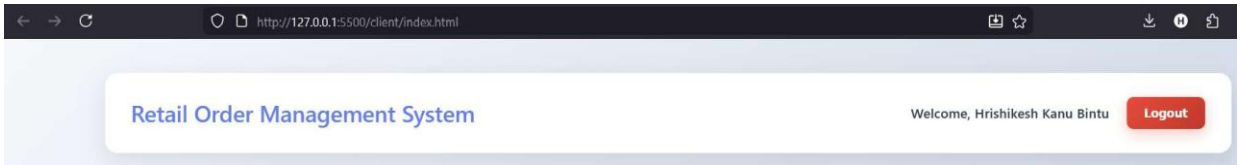

Figure 1.4: Website is running now on browser

```
D:\IIT\2.2\New folder\Project\retail-order-management-system\server>node server.js
Server running on http://localhost:8081
Connected to MongoDB
{
  userId: '699082cd29c3c1d35e4806b5',
  product_id: 3,
  product_name: 'Headphones',
  customer: 'Hrishikesh Kanu Bintu',
  quantity: 7,
  price: '31500.00',
  status: 'Completed',
  createdAt: 2026-02-14T14:48:29.501Z
}
Order inserted successfully: new ObjectId('69908b3d29c3c1d35e4806b6')
```

Figure 1.5: Ordered item details also available in the command prompt

1.4 Verification

To verify that the implementation is successful, the user should check the browser console (F12). A successful connection will show no "CORS" errors and will display the product data fetched from the backend. At this point, the system is fully operational and ready for user interaction.

Results And Discussion

2.1 Overview of the Final System

The **Retail Order Management System** was successfully designed, developed, and tested. The final application provides a fully functional web-based interface for users to register, log in, browse products, and manage their orders. All core modules—including the authentication system, database connectivity, and dynamic order processing—are operational and meet the initial design specifications. The system runs efficiently on the Node.js runtime environment, with real-time data persistence handled by MongoDB Atlas.

2.2 User Authentication Modules

Security and user management are foundational components of the **Retail Order Management System**. The authentication modules are engineered to control access strictly, ensuring that only authorized individuals can view product data, place orders, or access personal transaction history. The system employs industry-standard security practices, including password hashing and token-based authentication, to protect user integrity.

2.2.1 Registration Page

The Registration Module serves as the initial entry point for new users. It features a clean, intuitive interface designed to minimize user friction while gathering essential account data.

2.2.1.1 **Frontend Validation:** The interface employs client-side JavaScript validation to ensure data integrity before any request is sent to the server. It verifies that all required fields (Full Name, Email, Password) are populated and that the email follows a standard format.

2.2.1.2 **Backend Security:** Upon submission, the user's password is never stored in plain text. The backend utilizes the Bcrypt hashing algorithm to encrypt the password with a unique salt. This ensures that even in the event of a database compromise, user credentials remain protected against unauthorized decryption.

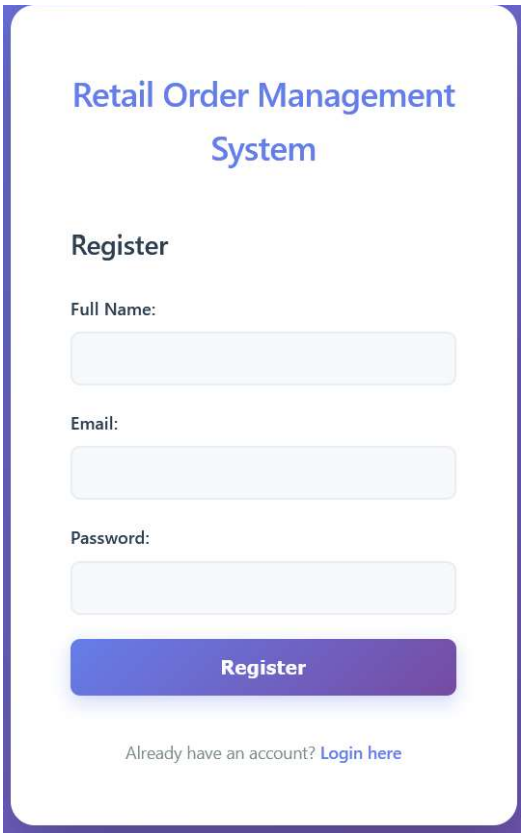

Figure 2.1: Registration Form

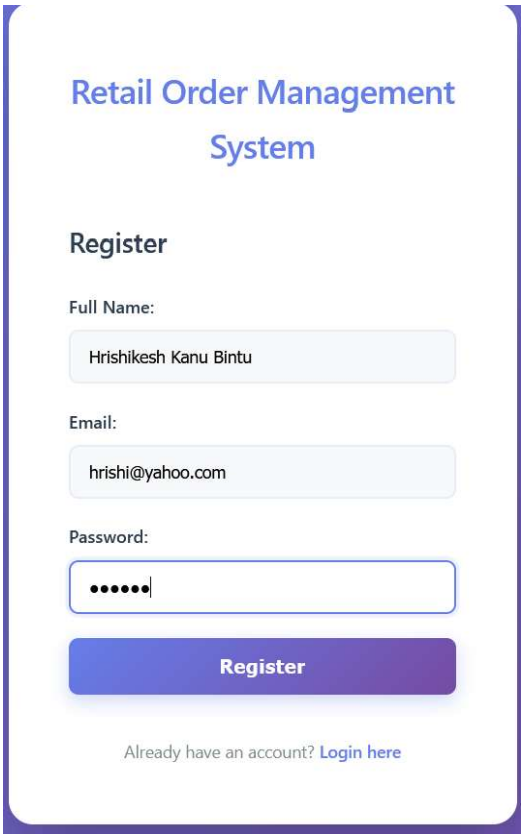

Figure 2.2: Registration Form (Filled)

2.2.2 Login Page

The Login Module provides a secure gateway for returning users. It authenticates credentials against the encrypted data stored in the MongoDB database.

2.2.2.1 **JWT Implementation:** The system uses **JSON Web Tokens (JWT)** for stateless authentication. When a user successfully logs in with their email and password, the server generates a signed token containing the user's ID and expiration time.

2.2.2.2 **Session Security:** Instead of relying on traditional server-side sessions, this token is securely transmitted to the client and stored in the browser's localStorage. For every subsequent request (such as placing an order), this token is sent in the HTTP header, allowing the server to verify the user's identity continuously without requiring repeated logins.

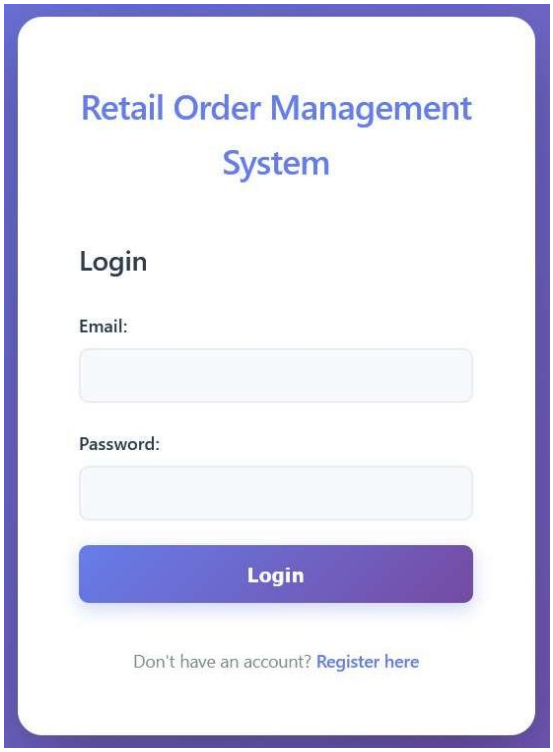

Figure 2.3: Login Page

2.3 Main System Features

Upon successful authentication, users are granted access to the core functional modules of the application. These features are integrated into a responsive, single-page dashboard designed to maximize usability and efficiency. The interface serves as the primary workspace where users can interact with the product inventory, manage their shopping activities, and review their transaction history in real-time.

2.3.1 User Dashboard

The User Dashboard acts as the central hub of the application. It employs a modern "Glassmorphism" design philosophy, utilizing translucent backgrounds and blur effects to create a visually distinct and hierarchical layout.

2.3.1.1 **Personalization:** The dashboard features a personalized greeting system (e.g., "Welcome, [User Name]"). This is achieved by decoding the payload of the stored **JWT token** on the client side, extracting the user's name without needing an additional database query.

2.3.1.2 **Layout:** The interface is divided into logical sections navigation, product selection, and order history ensuring that all critical tools are immediately accessible without navigation fatigue.

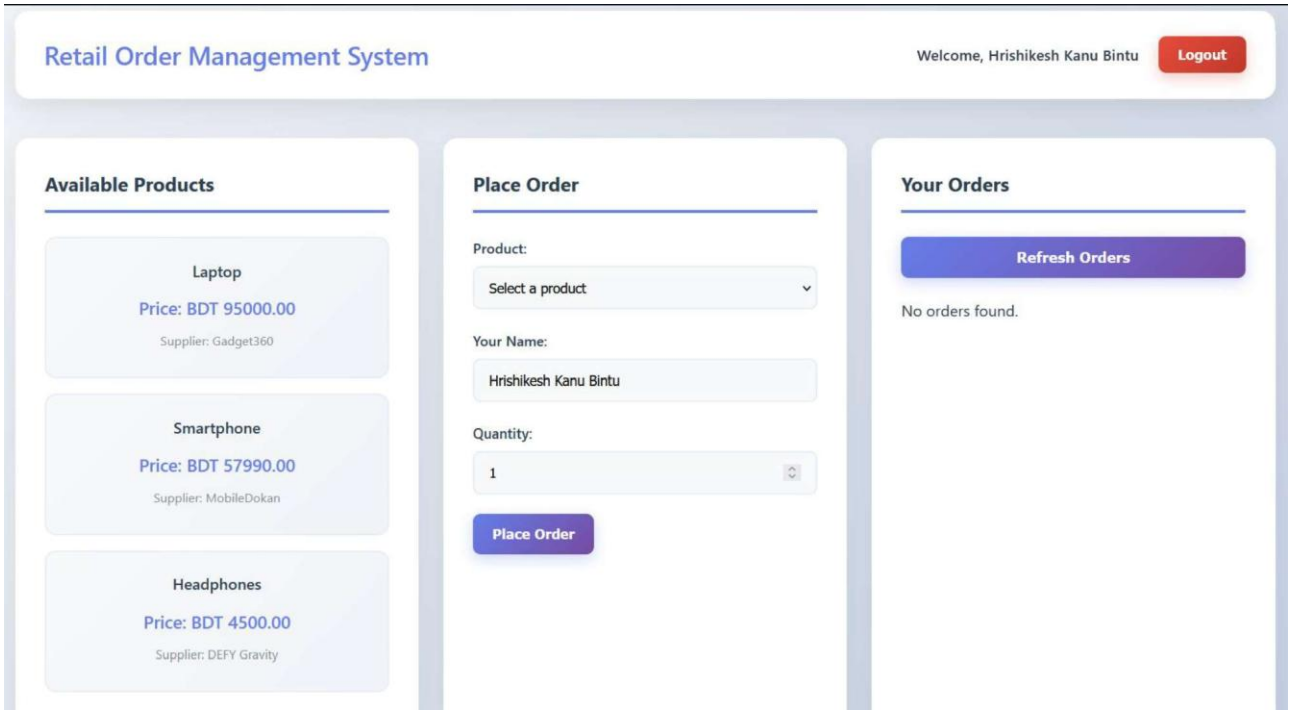

Figure 2.4: Main Dashboard

2.3.2 Product Catalog & Available Products

The Product Catalog module is the heart of the ordering system. Unlike static HTML lists found in simpler websites, this section is dynamic and real-time.

2.3.2.1 **Real-Time Data Fetching:** When the dashboard loads, the frontend automatically initiates a GET request to the backend API (/products). The server retrieves the active product—list including names, IDs, and unit prices directly from the **MongoDB products collection**.

2.3.2.2 **Dynamic Rendering:** The frontend script takes this JSON data and dynamically populates the "Select Product" dropdown menu.

2.3.2.3 **Inventory Control:** This real-time linkage ensures that users can only select products that are currently defined and available in the system. If a product is removed from the database by an

admin, it immediately disappears from the dashboard for all users, effectively preventing errors related to out-of-stock items.

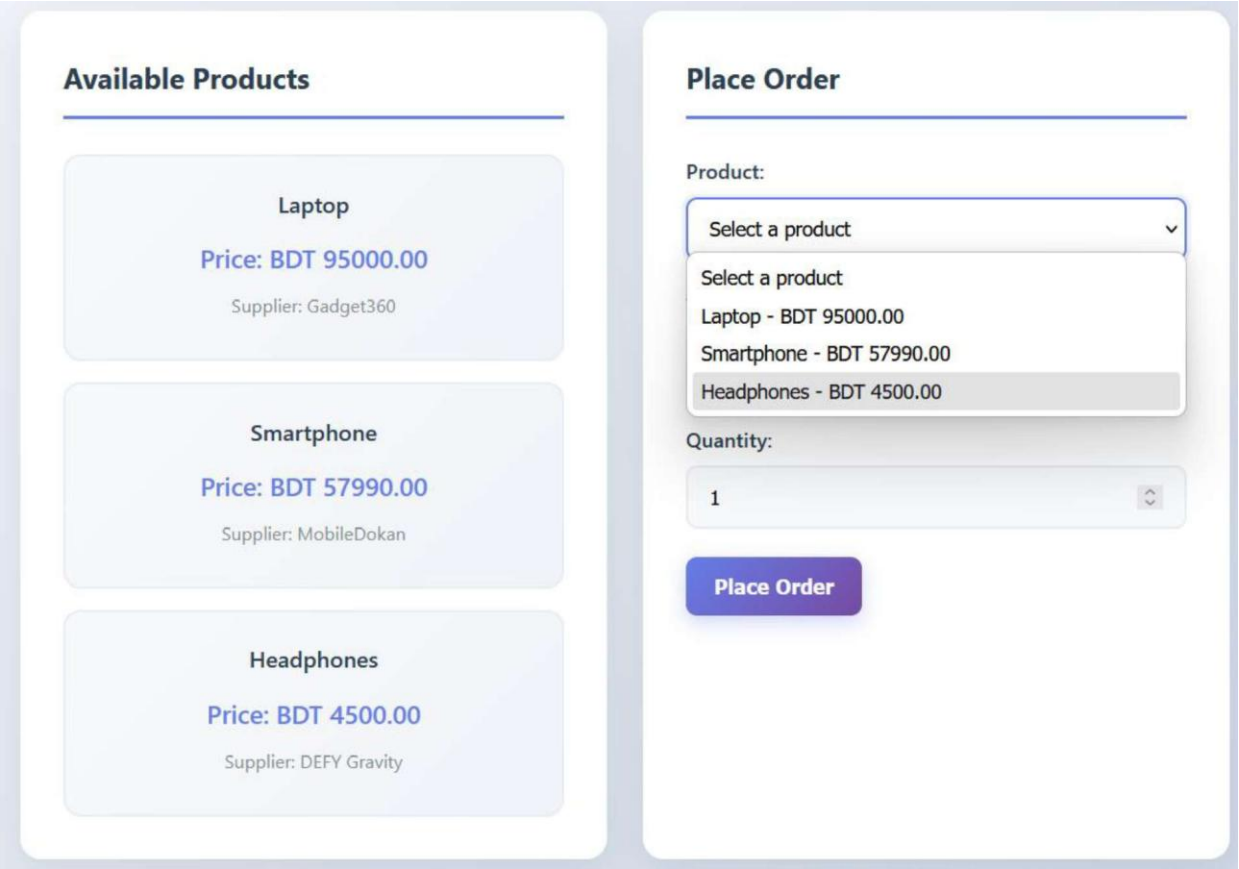

Figure 2.5: Product Dropdown Menu

2.3.3 Order Placement Module

The Order Placement feature is designed to streamline the complex process of creating a transaction into a few simple clicks. It abstracts the financial logic away from the user, ensuring accuracy and speed.

2.3.3.1 **User Interaction:** The user simply selects a valid product from the catalog and enters the desired quantity in the input field.

2.3.3.2 **Backend Logic & Security:** Upon clicking the "Place Order" button, the system executes a secure **POST** request to the /order endpoint. The backend server then performs a sequence of critical operations:

2.3.3.2.1 **Token Verification:** It validates the user's session token to ensure the request is authorized.

2.3.3.2.2 **Price Retrieval:** It looks up the current unit price of the selected product from the database (preventing price manipulation from the client side).

2.3.3.2.3 **Cost Calculation:** It automatically calculates the **Total Cost** (Unit Price × Quantity) and saves the complete transaction record to the orders collection.

2.3.3.2.4 **Instant Feedback:** Finally, it sends a success response to the frontend, which immediately updates the Order History table without refreshing the page.

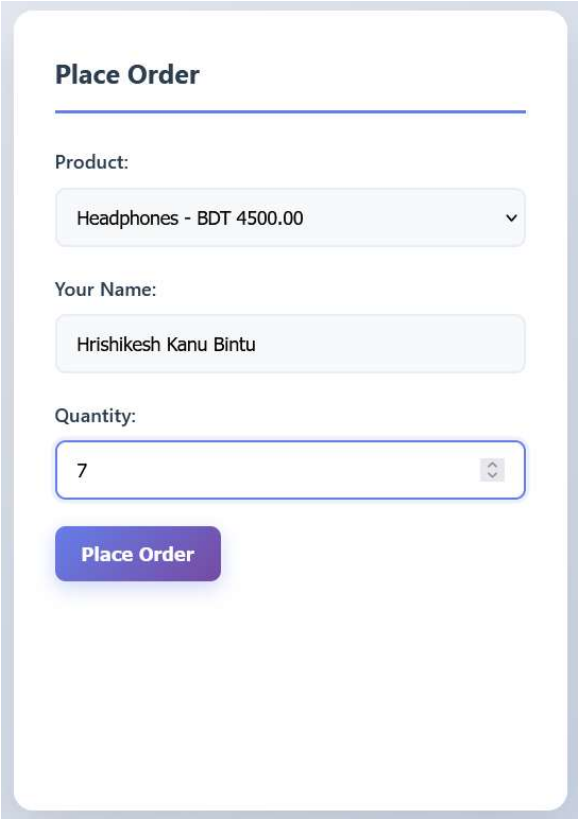

Figure 2.6: Placing an order

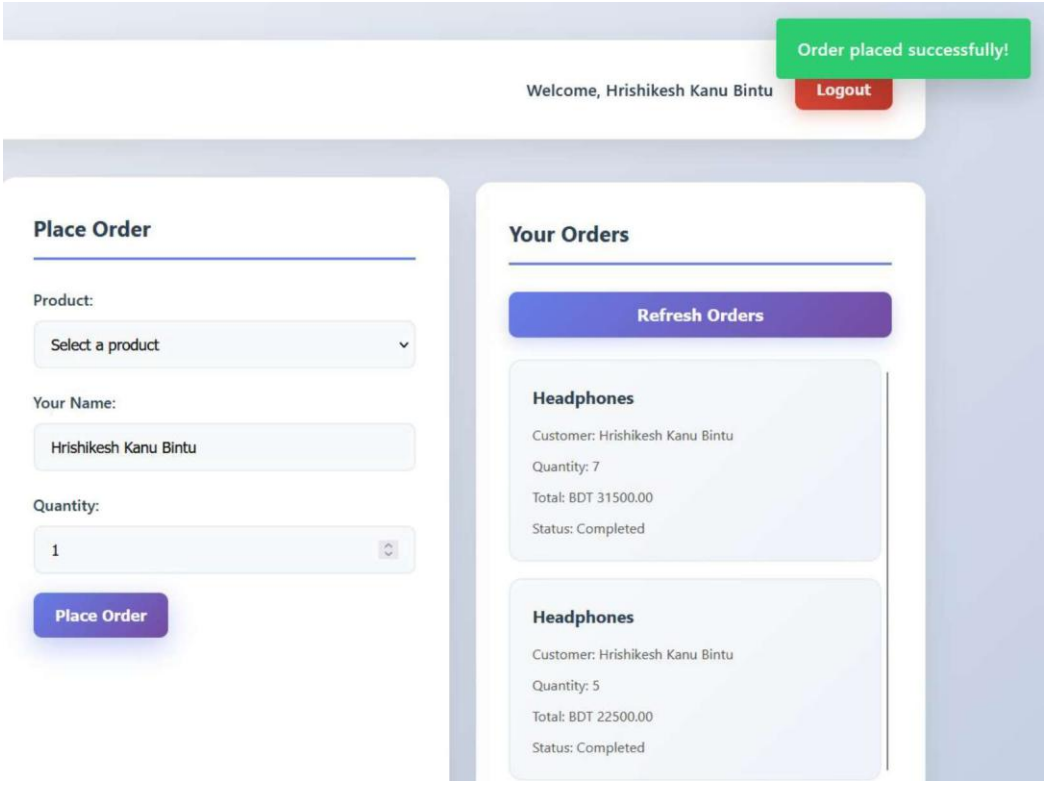

Figure 2.7: Order Placed Successfully

2.4 System Outputs

The system is designed to provide immediate visual feedback to the user and maintain a persistent, organized record of all transactions on the server.

2.4.1 Order History Table

The Order History module is the primary output interface for the end-user. Located immediately below the order placement form on the dashboard, this table displays a chronological list of all transactions associated with the logged-in account.

2.4.1.1 **Dynamic Rendering:** Upon loading the dashboard, the frontend executes a GET request to the

/orders endpoint. The server authenticates the request using the session token and returns a JSON array of orders, which is then dynamically rendered into HTML table rows.

2.4.1.2 **Data Visibility:** The table presents critical transaction details including the Order ID, Product Name, Quantity Purchased, Total Price (calculated server-side), and the current Order Status. This allows users to track their spending and order fulfillment status in real-time.

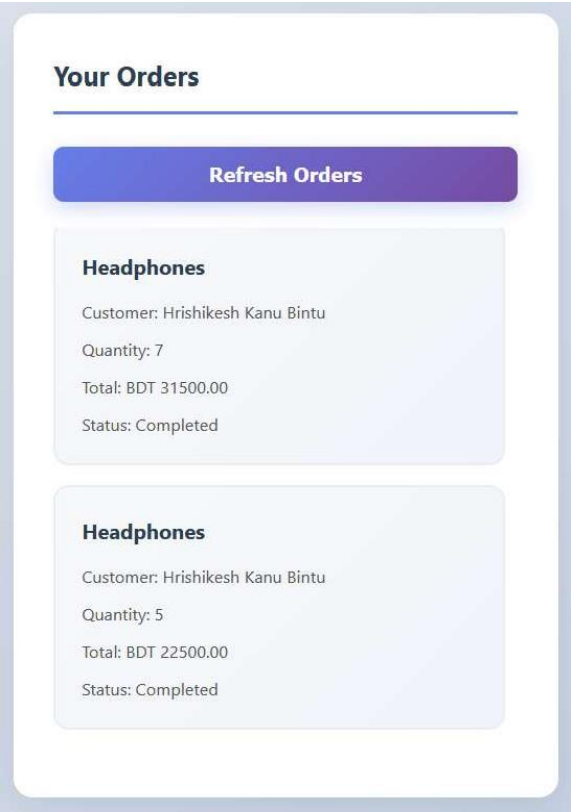

Figure 2.8: Order History

2.4.2 Database Records (Backend View)

While invisible to the standard frontend user, the **MongoDB Atlas** database serves as the authoritative source of truth for the system. The database is organized into two primary collections—users and orders—which store the raw data powering the application.

2.4.2.1 **User Management (users collection):** This collection stores all registered account information. As seen in the database records, each document contains the user's Name, Email, and a Hashed Password (encrypted via Bcrypt). This ensures that even database administrators cannot view sensitive user credentials in plain text.

2.4.2.2 **Order Transaction Records (orders collection):** This collection functions as the digital ledger for the business. Each order is stored as a distinct document containing:

2.4.2.2.1 **User ID:** A reference linking the order to the specific customer.

2.4.2.2.2 **Product Details:** The name and quantity of the item purchased.

2.4.2.2.3 **Financial Data:** The total cost calculated by the server.

2.4.2.2.4 **Timestamp:** The exact date and time the order was placed.

Data Integrity: The NoSQL structure allows these two collections to interact efficiently. The system can quickly query the orders collection to find all purchases made by a specific user_id, ensuring accurate history tracking and scalability as the volume of data increases.

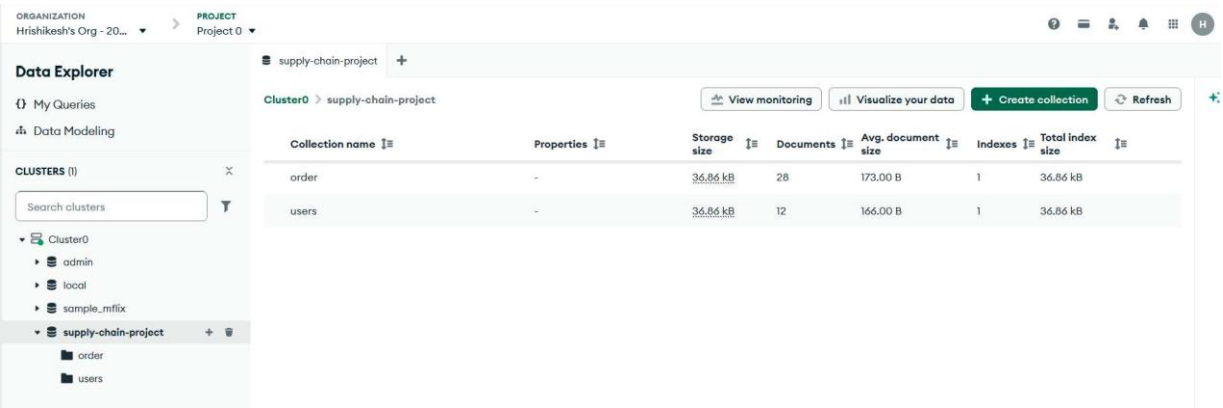

Figure 2.9: MongoDB Atlas Project Collection

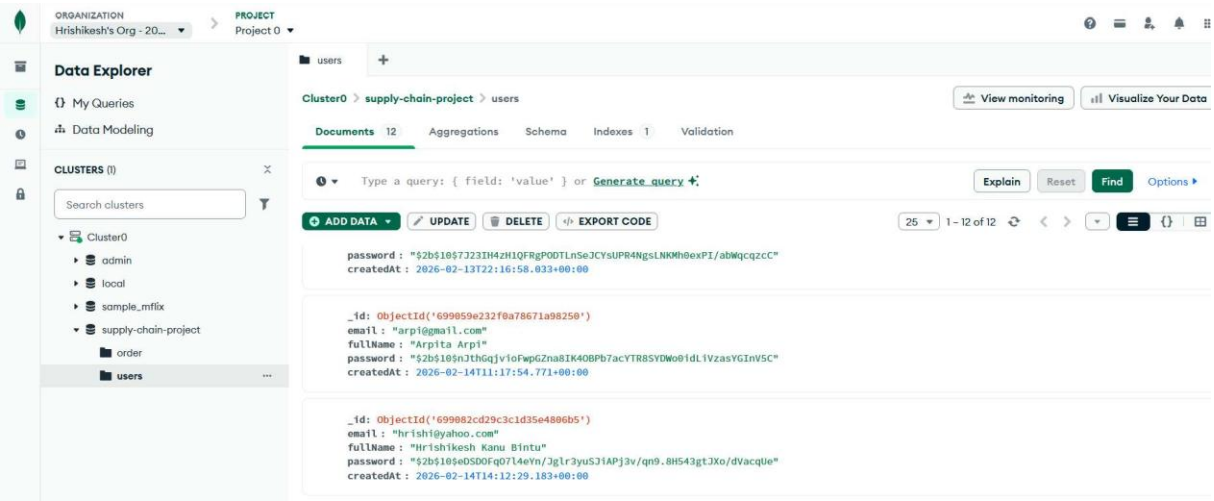

Figure 2.10: MongoDB Atlas Project Collection (Users)

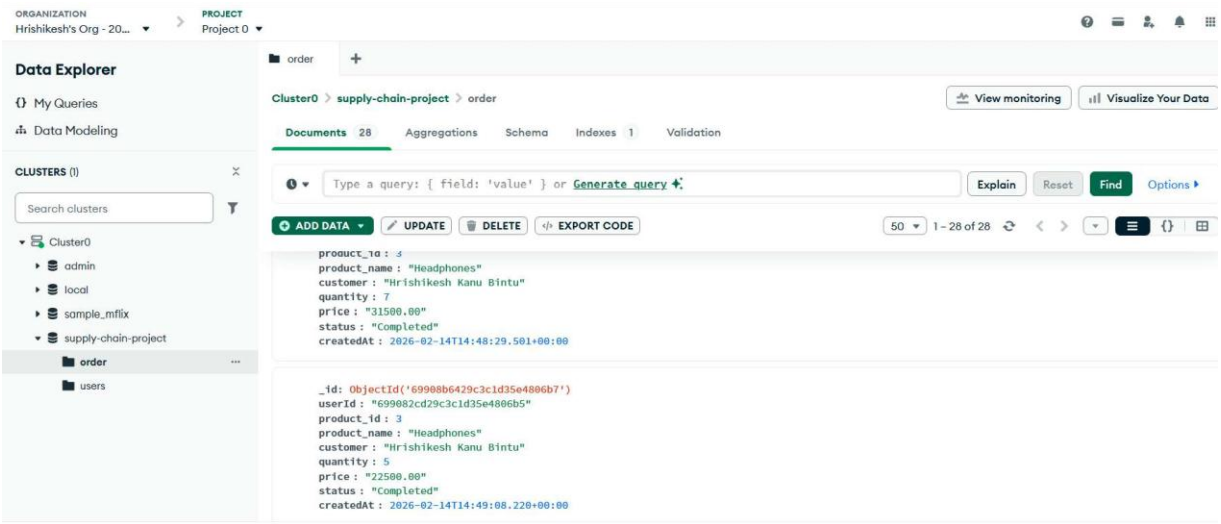

Figure 2.11: MongoDB Atlas Project Collection (Orders)

2.5 Discussion

The development and implementation of the **Retail Order Management System** have successfully met the core objectives outlined at the beginning of this project. The system demonstrates a significant improvement over manual tracking methods in three key areas:

2.5.1 Automation of Financial Logic

One of the primary goals was to eliminate human error in billing. The system effectively achieves this by automating the price calculation logic on the backend.

> **Result:** By strictly calculating Total = Unit Price × Quantity on the server side, the application prevents client-side manipulation and ensures that every recorded transaction is mathematically accurate. This replaces error-prone manual entry methods often found in legacy paper-based systems.

2.5.2 Enhanced Security Implementation

The integration of Bcrypt for password hashing and JSON Web Tokens (JWT) for session management has created a robust security layer that meets modern web standards.

> **Result:** Testing confirmed that unauthorized users are strictly blocked from accessing the dashboard or placing orders. Furthermore, the use of protected API endpoints (verifyToken middleware) ensures that sensitive user credentials and order history are never exposed to public requests.

2.5.3 Operational Efficiency & UX

The application demonstrates high responsiveness due to its Asynchronous Architecture and Glassmorphism UI.

Result: The use of the Fetch API allows for data retrieval and order placement without requiring full page reloads. This significantly reduces network latency and improves the overall user experience (UX), making the order management process faster and more fluid compared to traditional static web forms.